

FC7xxx MCU Integration

Manual

Rev.0.3

Revision History

Revision	Date	Changes
0.1	2023/07/14	Initial release for MCAL V0.1.0
0.2	2025/01/15	Update for adding more platform support
0.3	2025/10/10	Update for adding more platform support

Table of Contents

Revision History	2
Table of Contents	3
Chapter 1 Introduction	4
1.1 Introduction	4
Chapter 2 Building	5
2.1 Dependencies on Other Modules	5
2.2 Files Required for Compile	5
2.3 Add Plug-ins	6
Chapter 3 Memory	7
3.1 Sections in Memory Map	7
Chapter 4 Interrupt Service Routine (ISR).....	9
Chapter 5 Error Report	10
5.1 Det.....	10
5.2 Dem.....	10
Chapter 6 Function Calls.....	11
6.1 Function Calls during Startup	11
6.2 Function Calls during Shutdown.....	11
6.3 Function Calls during Sleep.....	11
6.4 Function Calls during Wake-up.....	11
6.5 Function Calls during Runtime	11
Chapter 7 Other Requirements.....	12
7.1 Notification, Callback, Callout	12
7.2 Macros.....	12
7.3 Inline assembly language.....	13
Chapter 8 Integration Steps.....	14

Chapter 1 Introduction

1.1 Introduction

This integration manual describes the integration requirements for the MCU module.

Chapter 2 Building

2.1 Dependencies on Other Modules

Module configuration dependency

- DEM: This module provides MCU DEM event parameter reference. For EB BSW we can easily add link to DEM Event. For other integration SW development tool, we may need to manually change code to link to DEM Report function.
- Common: This module is the basic module which used to choose the chip variant.

Module build dependency

- Common: This module provides common type for other modules.
- Det: This module is necessary for enabling Development error detection.
- Dem: This module is necessary for DEM event report error status.

Module initialization dependency

- Mcu is the basic module for other module configurations.

2.2 Files Required for Compile

MCU module files:

- _MCAL/Src/Mcu/Src/Mcu.c
- _MCAL/Src/Mcu/Src/Mcu_Hw.c
- _MCAL/Src/Mcu/Src/Mcu_Wku.c
- _MCAL/Src/Mcu/Src/Mcu_Scm.c
- _MCAL/Src/Mcu/Src/Mcu_Csc.c
- _MCAL/Src/Mcu/Src/Mcu_Smc.c
- _MCAL/Src/Mcu/Src/Mcu_Scg.c
- _MCAL/Src/Mcu/Src/Mcu_Rgm.c
- _MCAL/Src/Mcu/Src/Mcu_Pmc.c
- _MCAL/Src/Mcu/Src/Mcu_Pcc.c
- _MCAL/Src/Mcu/Src/Mcu_Cmu.c
- _MCAL/Src/Mcu/Src/Mcu_Irq.c

- _MCAL/Src/Mcu/include/Mcu.h
- _MCAL/Src/Mcu/include/Mcu_Hw.h
- _MCAL/Src/Mcu/include/Mcu_Types.h
- _MCAL/Src/Mcu/include/Mcu_Hw_Types.h
- _MCAL/Src/Mcu/include/Mcu_MemMap.h
- _MCAL/Src/Mcu/include/Mcu_Version.h
- _MCAL/Src/Mcu/include/Mcu_Wku.h
- _MCAL/Src/Mcu/include/Mcu_Wku_RegOps.h
- _MCAL/Src/Mcu/include/Mcu_Wku_Reg.h
- _MCAL/Src/Mcu/include/Mcu_Csc.h
- _MCAL/Src/Mcu/include/Mcu_Csc_RegOps.h
- _MCAL/Src/Mcu/include/Mcu_Scm.h
- _MCAL/Src/Mcu/include/Mcu_Scm_RegOps.h
- _MCAL/Src/Mcu/include/Mcu_Smc_RegOps.h
- _MCAL/Src/Mcu/include/Mcu_Smc_Reg.h

- _MCAL/Src/Mcu/include/Mcu_Smc.h
- _MCAL/Src/Mcu/include/Mcu_Cmu_RegOps.h
- _MCAL/Src/Mcu/include/Mcu_Cmu_Reg.h
- _MCAL/Src/Mcu/include/Mcu_Cmu.h
- _MCAL/Src/Mcu/include/Mcu_Pmc_RegOps.h
- _MCAL/Src/Mcu/include/Mcu_Pmc_Reg.h
- _MCAL/Src/Mcu/include/Mcu_Pmc.h
- _MCAL/Src/Mcu/include/Mcu_Rgm_RegOps.h
- _MCAL/Src/Mcu/include/Mcu_Rgm_Reg.h
- _MCAL/Src/Mcu/include/Mcu_Rgm.h
- _MCAL/Src/Mcu/include/Mcu_Scg_RegOps.h
- _MCAL/Src/Mcu/include/Mcu_Scg_Reg.h
- _MCAL/Src/Mcu/include/Mcu_Scg.h
- _MCAL/Src/Mcu/include/Mcu_Pcc_RegOps.h
- _MCAL/Src/Mcu/include/Mcu_Pcc.h
- _MCAL/Src/Mcu/include/Mcu_CM7_RegOps.h
- _MCAL/Src/Mcu/include/Mcu_CM7_Reg.h

MCU Generate files

- _MCAL_generate/src/Mcu_PBcfg.c
- _MCAL_generate/src/Mcu_Cfg.c
- _MCAL_generate/include/Mcu_Cfg.h
- _MCAL_generate/include/Mcu_CfgPlatformDef.h

Common module files:

- _MCAL/Src/Common/include/Mcal.h
- _MCAL/Src/Common/include/Std_Types.h
- _MCAL/Src/Common/include/Platform_Types.h
- _MCAL/Src/Common/include/Compiler.h
- _MCAL/Src/Common/include/Compiler_Cfg.h
- _MCAL/Src/Common/include/CompilerDefinition.h

Det module files:

- Det.h

2.3 Add Plug-ins

MCU module plug-ins are developed for EB tresos Studio, so, to use MCU plug-ins on the EB tresos Studio, the user needs to add the MCU module to the EB plug-ins folder first.

- 1) Copy Mcu module(_MCAL/EB_Plugins/eclipse/plugins/Mcu) folder to EB tresos plug-ins (EB/tresos/plugins/) folder.
- 2) Set Mcu module output location folder for the generated source file and header files.
- 3) Use EB tresos to configure Mcu module and generate source and header files.

Chapter 3 Memory

3.1 Sections in Memory Map

Section Name	Section Type	Description
MCU_START_SEC_CONFIG_DATA_8 MCU_STOP_SEC_CONFIG_DATA_8 MCU_START_SEC_CONFIG_DATA_16 MCU_STOP_SEC_CONFIG_DATA_16 MCU_START_SEC_CONFIG_DATA_32 MCU_STOP_SEC_CONFIG_DATA_32	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are initialized by startup code(data).
MCU_START_SEC_CONFIG_DATA_UNSPECIFIED MCU_STOP_SEC_CONFIG_DATA_UNSPECIFIED	Configuration Data	Start and stop of Memory Section for Config Data.
MCU_START_SEC_CONST_BOOLEAN MCU_STOP_SEC_CONST_BOOLEAN MCU_START_SEC_CONST_8 MCU_STOP_SEC_CONST_8 MCU_START_SEC_CONST_16 MCU_STOP_SEC_CONST_16 MCU_START_SEC_CONST_32 MCU_STOP_SEC_CONST_32 MCU_START_SEC_CONST_UNSPECIFIED MCU_STOP_SEC_CONST_UNSPECIFIED	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit or boolean. These variables are read-only (rodata).
MCU_START_SEC_CODE MCU_STOP_SEC_CODE MCU_START_SEC_RAMCODE MCU_STOP_SEC_RAMCODE MCU_START_SEC_CODE_AC MCU_STOP_SEC_CODE_AC	Code	Start and stop of memory Section for Code (text).
MCU_START_SEC_VAR_NO_INIT_BOOLEAN MCU_STOP_SEC_VAR_NO_INIT_BOOLEAN MCU_START_SEC_VAR_NO_INIT_8 MCU_STOP_SEC_VAR_NO_INIT_8 MCU_START_SEC_VAR_NO_INIT_16 MCU_STOP_SEC_VAR_NO_INIT_16 MCU_START_SEC_VAR_NO_INIT_32 MCU_STOP_SEC_VAR_NO_INIT_32 MCU_START_SEC_VAR_NO_INIT_UNSPECIFIED MCU_STOP_SEC_VAR_NO_INIT_UNSPECIFIED	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are never cleared and never initialized by startup code (bss).
MCU_START_SEC_VAR_INIT_BOOLEAN MCU_STOP_SEC_VAR_INIT_BOOLEAN MCU_START_SEC_VAR_INIT_8 MCU_STOP_SEC_VAR_INIT_8 MCU_START_SEC_VAR_INIT_16 MCU_STOP_SEC_VAR_INIT_16 MCU_START_SEC_VAR_INIT_32 MCU_STOP_SEC_VAR_INIT_32	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are initialized by startup code (data).

Section Name	Section Type	Description
MCU_START_SEC_VAR_INIT_UNSPECIFIED		
MCU_STOP_SEC_VAR_INIT_UNSPECIFIED		

Chapter 4 Interrupt Service Routine (ISR)

Instance	Interrupt Name	IRQ Number (NVIC Interrupt ID)
CMU0	Mcu_CMU0_ISR	53
CMU1	Mcu_CMU1_ISR	54
CMU2	Mcu_CMU2_ISR	55
CMU3	Mcu_CMU3_ISR	56
CMU4	Mcu_CMU4_ISR	153
RGM	MCU_RGM_ISR	47
SCG	MCU_SCG_ISR	52
PMC	Mcu_PMC_LVD_LVW_ISR	35

Chapter 5 Error Report

5.1 Det

Function Name	Error Type
Mcu_Init	MCU_E_PARAM_CLOCK
Mcu_InitRamSection	MCU_E_UNINIT; MCU_E_PARAM_RAMSECTION
Mcu_InitClock	MCU_E_UNINIT; MCU_E_PARAM_POINTER; MCU_E_PARAM_CLOCK;
Mcu_SetMode	MCU_E_UNINIT; MCU_E_PARAM_MODE;
Mcu_DistributePllClock	MCU_E_UNINIT; MCU_E_PLL_NOT_LOCKED
Mcu_GetPllStatus	MCU_E_UNINIT;
Mcu_GetResetReason	MCU_E_UNINIT;
Mcu_GetResetRawValue	MCU_E_UNINIT;
Mcu_PerformReset	MCU_E_UNINIT;
Mcu_GetVersionInfo	MCU_E_PARAM_POINTER
Mcu_GetRamState	MCU_E_UNINIT;
Mcu_DeInitClock	MCU_E_PARAM_POINTER;
Mcu_DisablePmc	MCU_E_UNINIT;
Mcu_DisableRgm	MCU_E_UNINIT;
Mcu_GetSoscValidStatus	MCU_E_UNINIT;

5.2 Dem

Function name	Error type
Mcu_InitClock	MCU_E_CLOCK_FAILURE

Chapter 6 Function Calls

6.1 Function Calls during Startup

The APIs that need to be called are:

- `void Mcu_Init(Mcu_ConfigType *pConfigPtr)`
- `Std_ReturnType Mcu_InitClock(Mcu_ClockType ClockSetting).`

Optional:

- `Std_ReturnType Mcu_InitRamSection(Mcu_RamSectionType RamSection) .`
- `Mcu_PllStatusType Mcu_GetPllStatus(void)`
- `Std_ReturnType Mcu_DistributePllClock(void)`

6.2 Function Calls during Shutdown

None

6.3 Function Calls during Sleep

The `Mcu_SetMode` API will enable WKU and close the CMU instance when the CMU is not enabled to run in stop or run in low power.

If the CMU instance is not disabled, the CMU will generate a clock monitor error and the interrupt function will be called after wake-up in standby mode or wake-up in stop mode.

6.4 Function Calls during Wake-up

When using CMU, the user needs to enable CMU by using `Mcu_EnableCmu` API.

6.5 Function Calls during Runtime

The APIs that need to be called are:

- `void Mcu_SetMode(Mcu_ModeType McuMode)`
- `void Mcu_PerformReset(void)`

These APIs need to be called at runtime, but would trigger an ECU mode change.

Chapter 7 Other Requirements

7.1 Notification, Callback, Callout

- Notification

If the user configures notifications and the value is not "NULL_PTR" or "NULL", the user shall implement the notification function in the user code.

Example:

```
void Cmu_Notification(uint8 eInstance)
```

```
void Pmc_Notification(void)
```

```
void Rgm_Notification(uint32 u32SRS)
```

- Callback

None

- Callout

None

7.2 Macros

Please check various definitions available in Common module's include file Mcal.h for details.

- If OS is not used:

AUTOSAR_OS_NOT_USED need defined.

In this case, If USE_SW_VECTOR_MODE is not defined, the user needs to map the interrupt vector table function to ISR function, for example:

```
typedef void (*CMU_pInterruptNotify)(uint8 u8Instance);
```

```
CMU_pInterruptNotify CMU_ErrorNotify = NULL_PTR;
```

```
#if MCU_CMU_ISR_NOTIFICATION_USED
```

```
    if(MCU_CMU_NOTIFICATION != NULL_PTR)
```

```
    {
```

```
        CMU_ErrorNotify = MCU_CMU_NOTIFICATION;
```

```
    }
```

```
#endif
```

```
ISR(Mcu_CMU0_ISR)
```

```
{
```

```
    CMU_IRQHandler(CMU_INSTANCE_0);
```

```
    if (CMU_ErrorNotify != NULL_PTR)
```

```
{
```

```
    CMU_ErrorNotify(CMU_INSTANCE_0);  
}  
  
EXIT_INTERRUPT();  
}
```

- If OS is used:
Do not define AUTOSAR_OS_NOT_USED.

7.3 Inline assembly language

- `__asm( "wfi" )`

This inline assembly code is used to let the arm core enter sleep mode.

Used by Mcu_Smc.c

Chapter 8 Integration Steps

- 1) Configure the MCU module and generate configuration files (please refer to Building chapter for details).
- 2) Configure appropriate memory sections in linker file or other (please refer to Memory chapter for details).
- 3) Map interrupt notification to their vector locations (please refer to chapter ISR for details).
- 4) Build the MCU module with dependent modules.